

A OPTIONATED IMPLEMENTATION OF

Harness Engineering

Making the Repository itself the First-class Citizen of Harness

Five Subsystems for Harness

The repository is the specification. Miss any one subsystem and the harness is incomplete.

1

Instruction

AGENTS.md / CLAUDE.md:

Overview + a Map

AGENTS.md

2

Tool

Sufficient shell + file access
under least privilege.

Async and sequence tools to
avoid race conditions.

least privilege

3

Environment

Self-describing, locked,
reproducible deps.

Makefile · pyproject.toml ·
Dockerfile

4

State

PROGRESS.md: what's done, in
progress, blocked. Updated
before each session ends.

PROGRESS.md

5

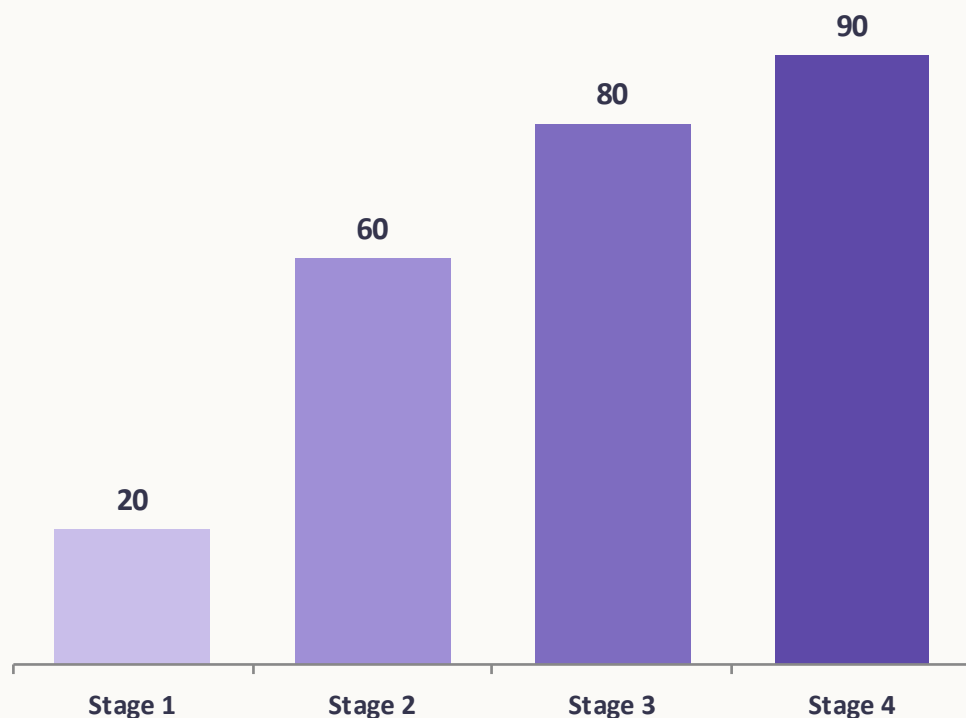
Feedback

Explicit verification commands
— the highest-ROI subsystem.

make contact-driven

Same Model, Four Iterations: 20% → ~100%

A ~20,000-line TypeScript + React app on GPT-4o. The model never changed — only the harness.



Stage 4 stabilized at 80–100%

20%

Stage 1 — README only

Failures: wrong package manager (npm vs yarn), broke naming conventions, couldn't run tests.

60%

Stage 2 — + AGENTS.md

Added tech-stack versions, naming conventions, key architecture decisions.

80%

Stage 3 — + verification commands

Listed `yarn test && yarn lint && yarn build` so the agent could check itself.

80–100%

Stage 4 — + progress templates

Agent recorded completed / incomplete work every run.

You did not switch to a better model — what changed was the harness.

Cache Hit = Performance and Money

"Prompt caching is the architectural constraint around which the entire product is built."

Thariq Shihpar (Claude Code engineer)

↓ Latency

cached tokens aren't recomputed
Faster time to first token.

↓ Cost

cached input is billed at ~10x cheaper

A fresh session that reads the **same context** reuses the warm prefix instead of paying to rebuild context from scratch.

warm prefix
(cache hit)

System prompt

stable · cached

AGENTS.md + repo map

stable · cached

cache boundary

PROGRESS.md / task state

*volatile ·
recomputed*

Breaking the cache:

edit anything early — system prompt, AGENTS.md, file read-order, even whitespace — and everything downstream is recomputed.

Cache miss = slower + costlier.

The Fresh-Session Test

Open a blank session with only the repo. Can it answer five questions — and start work without asking a human?



The more hidden the information, the higher the discovery cost, and the less context budget remains for the actual task, thus lower quality.

How to Draw a Good Map

Four principles for documentation the agent can actually navigate.

1 Knowledge lives next to code

A rule about API auth belongs beside the API code — **reveal on-demand short doc** in each module directory explaining that module's responsibilities, interfaces, and constraint.

2 One standardized entry file

AGENTS.md / CLAUDE.md is the landing page.
50 - 100 lines is enough, 200 lines max.

This is a DIRECTORY (not encyclopedia)

3 Minimal but complete

Don't duplicate type definitions, interface comments, or configuration explanations inside instruction files.

4 Update with code

Bind doc updates to the same commit.
A CI or pre-commit hook reminds agent

Concrete repo layout

```
project/
├─ AGENTS.md           # Overview + Map
├─ src/module_A
│   └─ api/ARCHITECTURE.md
│       └─ db/CONSTRAINTS.md # MUST / DONT
├─ PROGRESS.md         # 4-state control
└─ Makefile
```

Discovery cost — context budget burned just to find a key fact

Knowledge decay rate — share of repo knowledge going stale — the biggest enemy

Unknown Decision → Inconsistent Code + Duplicated Works

Treat the agent like a shift engineer whose short-term memory is wiped at the end of every session.

Re-decision

Last session chose option B after costly analysis. This session, blind to it, may reverse to option A on less information.

Duplicate work

Redoes finished work — or half-redoes it, hits a conflict with the existing implementation, and has to rework.

⚠ Context anxiety

Near token limits (~60% utilization) agents start rushing, skipping verification, and picking suboptimal solutions.

Drift

Each session's understanding of the goal shifts slightly. Deviations compound until the result is far from intent.

Verification gap

Prior pass/fail state wasn't recorded, so every session re-diagnoses the whole repo from scratch.

Primary metric — Rebuild Cost

15 min of aimless repo exploration

————→ **< 3 min** of targeted reading + validation

the time a fresh session needs to reach an executable state

The fix: persist state in git-tracked files — **PROGRESS.md** + **DECISIONS.md** — so the next shift picks up where the last left off.

The Persistence Toolkit: PROGRESS.md + DECISIONS.md

Don't rely on chat history. These two git-tracked files are the handoff between shifts.

PROGRESS.md

the handoff log

Current State

- Latest commit: abc1234 (feat: prefs endpoint)
- Test status: 42/43 passing
(test_pagination_edge_case failing)
- Lint: passing

Completed

- [x] User model and DB migration
- [x] Basic CRUD endpoints

In Progress

- [] Pagination (90% - edge case failing)

Next Steps

1. Fix pagination edge case bug
2. Add 'include deleted users' param

DECISIONS.md

the architectural guardrail

2024-01-15: Redis for prefs caching

- Decision: cache user prefs in Redis
- Reason: high read frequency (every API call), small data size
- Rejected alternative: PostgreSQL materialized view – high change frequency makes maintenance not worthwhile
- Constraint: TTL 5 min, active invalidation on write

- **PROGRESS must capture** commit hash, test status, completed vs in-progress, blockers, numbered next steps.
- **DECISIONS must capture** the choice, the rejected alternative (and why), and active constraints — so a new session can't “optimize away” a deliberate design.

Clock-In / Clock-Out Discipline

A mandatory routine, defined in AGENTS.md, so no session starts or ends loose.

CLOCK-IN — session start

1. Read PROGRESS.md — status, blockers, next steps
2. Read DECISIONS.md — don't reverse past choices
3. Run make check / npm test — confirm consistent state
4. Resume from the “Next Steps” list

CLOCK-OUT — before session end

1. Update PROGRESS.md — status, known issues, next steps
2. Run tests — log the final verification state
3. Commit all completed work to git (atomic commits)

Repos Rot: Knowledge Drift

*The default trajectory of an agent-maintained codebase is **entropy** — unless managed at every session.*

“***Agents copy patterns already present in the repository, even when those patterns are inconsistent or suboptimal.***

— OpenAI, five months of Codex experiments

Knowledge drift

Out-of-date documentation is actively dangerous — it confidently guides the agent the wrong way. Worse than no docs at all.

Knowledge decay rate

The share of repo knowledge that goes stale per unit of time. Documentation drifting out of sync with code is the biggest enemy.

The 5-Dimension Session-Exit Checklist

Code compiling isn't "done." The harness gates session completion on five non-negotiable criteria — encoded in CLAUDE.md.

DIMENSION	VERIFICATION ACTION	GOAL
1 Build status	Run compilation / build	Next session never inherits an unbuildable codebase
2 Test suite	Run the full local test suite (integration + e2e)	Guarantee new changes haven't introduced regressions
3 Progress tracking	Update feature list, subtask statuses, exit criteria	Cuts next-session startup diagnostics by 60–80%
4 Artifact hygiene	Delete temp logs, debug statements, TODO markers	Prevents cognitive slop and pattern-copying drift
5 Editable changes	Atomic commit (use worktree; --no-ff)	Immediate continuity without manual intervention

Dual-Mode Cleanup

Entropy management runs at two cadences — borrowed from garbage-collection theory.

Mode A — Immediate

end of every session · like reference-counting GC

- Clean up immediate side effects before exiting: temporary files, debug paths, local state.
- Update the feature-list state; ensure build + tests pass.
- Update module-level README.md and NOTE.md

Mode B — Periodic

weekly cadence, background · like tracing GC

- A fleet of background tasks scans for structural divergence and macro architecture-boundary issues. (High human involvement for now)
- Update quality scores, fix flaky tests, open + auto-merge targeted refactoring PRs.

Observability Belongs to the Harness

Without it, decisions are made under uncertainty, evaluation is subjective, and retries are blind wandering.

 **Do not rely on the agent to print its own logs or manage its own tracking.**

Agents don't know what they don't know.
Log formats are inconsistent across sessions.
Process signals can't be solved by print statements.

Five system-layer signals the harness must capture externally:



Application lifecycle states

startup → ready → running → shutdown



Feature path execution

entry points, checkpoints, exits on critical paths



Data flow

clear inputs / outputs between components



Resource utilization

abnormal patterns, e.g. runaway memory



Full error context

entire stack traces + env state, not just strings

30–50%

of the next session is wasted re-diagnosing when observability is missing

Double-Layered Observability

Capture execution data and intent data at once — then feed evidence-backed failures into a retry loop.

Runtime observability

“what did the system do?”

logs · system traces · process events · health checks

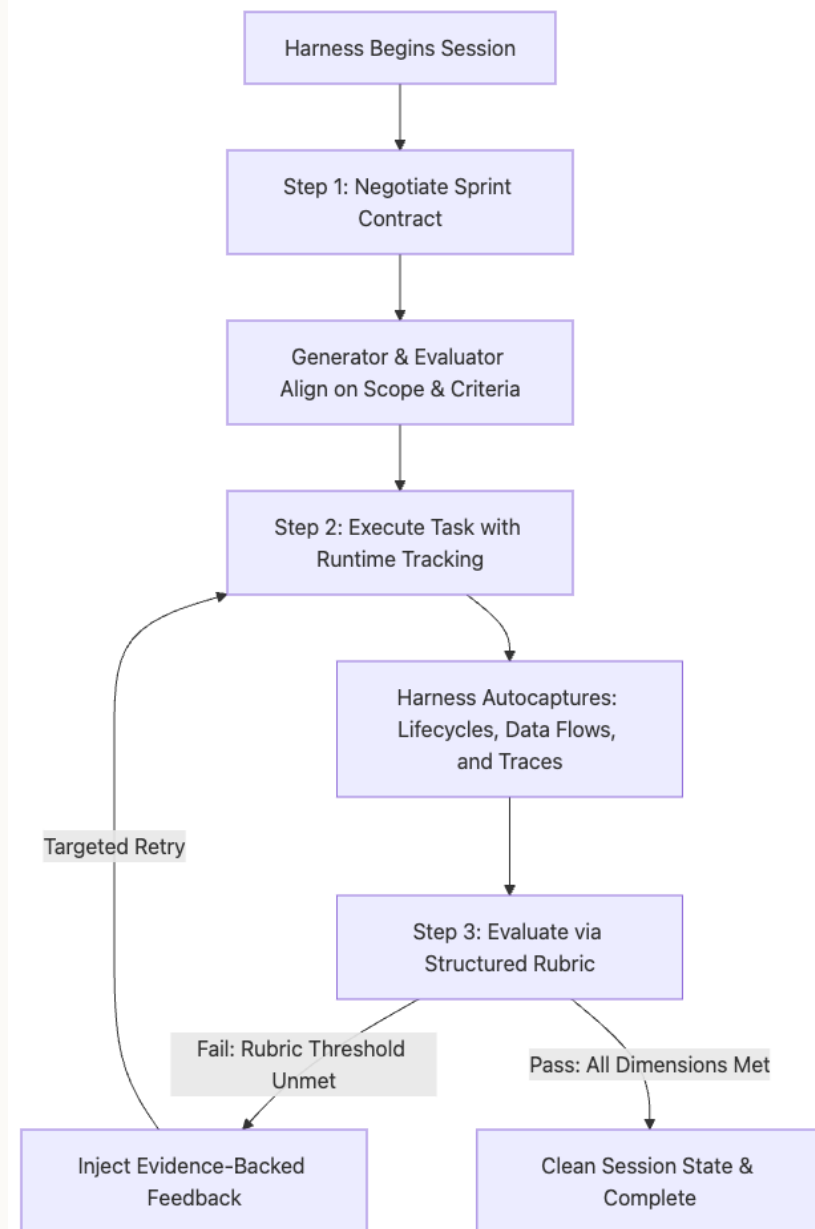
Process observability

“why should this change be accepted?”

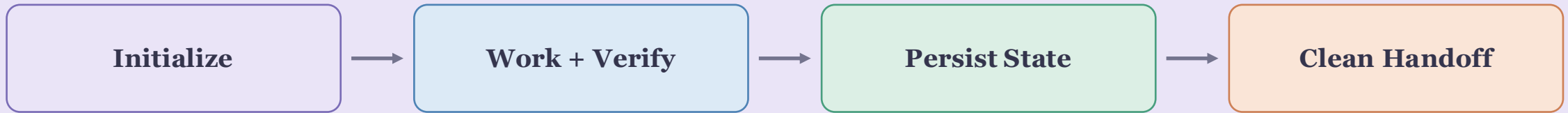
sprint contracts · upfront plans · scoring rubrics · acceptance criteria

Tracing Mapping Guide:

- Trace:** Map to a single complete Harness Session.
- Span:** Map to an individual Task or feature implementation block.
- Sub-Span:** Map to each individual verification, test run, or compilation step.



The Harness Lifecycle



↻ repeats every session, over the repo as single source of truth, with harness-layer observability throughout

- The repository is the specification — if it isn't in the repo, it doesn't exist to the agent.
- Five subsystems: Instruction, Tool, Environment, State, Feedback.
- Treat every session as a shift handoff — persist PROGRESS.md + DECISIONS.md.
- Verify by execution and observe at the harness layer, not the agent's self-report.
- Leave a clean handoff every time; prune the harness as models get stronger.